## 1. База данных. СУБД. Классификация

**База данных (1)** – совокупность взаимосвязанных данных некоторой предметной области, хранимых в памяти ЭВМ и организованных таким образом, что эти данные могут быть использованы для решения многих задач многими пользователями.

**База данных (2)** – документированное собрание интегрированных записей. Документированность говорит о наличии метаданных. Интегрированность говорит и собрании вместе файлов данных, индексов и метаданных приложения.

**Основные требования к БД**

1. **Неизбыточность** данных - каждое данное присутствует в БД в единственном экземпляре.
2. **Совместное использование** данных многими пользователями.
3. **Эффективность** доступа к БД - высокое быстродействие, т. е. малое время отклика на запрос.
4. **Целостность** данных - соответствие имеющейся в БД информации её внутренней логике, структуре и всем явно заданным правилам.
5. **Безопасность** данных – защита данных от преднамеренного или непреднамеренного искажения или разрушения данных.
6. **Восстановление** данных после программных и аппаратных сбоев.
7. **Независимость** данных от прикладных программ.

**OLAP** — online analytical processing (операционные данные) – чтение, выборка.

**OLTP** — online transaction processing (перманентные данные) – вставка, удаление, обновление.

**Система баз данных (СБД)** — совокупность одной или нескольких баз данных и комплекса информационных, программных и технических средств, обеспечивающих накопление, обновление, корректировку и многоаспектное использование данных в интересах пользователей.

**Система управления базами данных (СУБД)** - приложение, обеспечивающее создание, хранение, обновление и поиск информации в базах данных.

**Основные функции СУБД**

1. Управление данными во внешней памяти.
2) Управление буферами оперативной памяти.
3) Управление транзакциями.
4) Журнализация.
5) Поддержка языка или языкового пакета (-ов).

**Классификация СУБД**

**По модели данных**

1. Дореляционные.
    a. Инвертированные списки
    b. Иерархические
    c. Сетевые
2. Реляционные
3. Постреляционные

**По архитектуре организации данных**

1. Локальные
2. Распределенные

**По способу доступа к БД**

1. Файл-серверные (при работе с базой, данные перегоняются приложению, которое с ней работает, вне зависимости от того, сколько их нужно. Все операции — на стороне клиента. Файловый сервер периодически обновляется тем же клиентом)
2. Клиент-серверные (вся работа на сервере, по сети передаются результаты запросов, гораздо меньше информации. Обеспечивается безопасность данных, потому что все происходит на стороне сервера. Проще исключить одновременное изменение и тп)
3. Встраиваемые — библиотека, которая позволяет унифицированным образом хранить большие объемы данных на локальной машине. Доступ к данным может происходить через SQL либо через особые функции СУБД. Встраиваемые СУБД быстрее обычных клиент-серверных и не требуют установки сервера, поэтому востребованы в локальном ПО, которое имеет дело с большими объемами данных.
4. Сервисно-ориентированные (БД является хранилищем сообщений, промежуточных состояний, метаинформации об очередях сообщений и сервисах)

**Empty list**

## 2. Семантическое моделирование данных

**Мотивация**

1.Недостаточно средств для представления смысла данных

2.Сложно моделировать предметную область на основе плоских таблиц

3.Отсутствуют средства для представления зависимостей между данными

4.Нет аппарата для разделения сущностей и связей

Самый широко-распространенный вариант семантического моделирования – **Entity-Relationship Diagram в нотачии Чена**

**Основными понятиями ER нотации являются:**

1.**Сущность** - реальный или представляемый объект, информация о котором должна сохраняться и быть доступна. В диаграммах изображается прямоугольником с именем сущности. Различают:

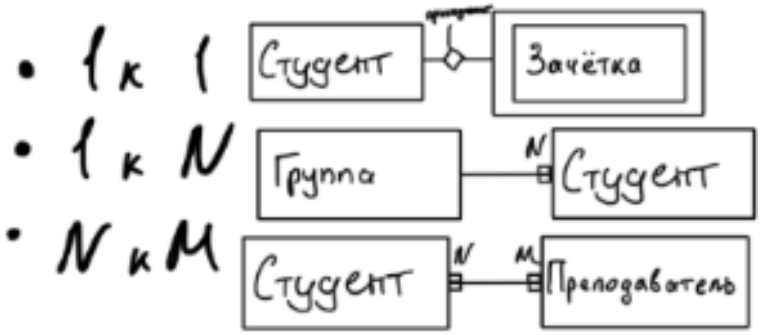
a. Сильные сущности (существуют сами по себе)
b. Слабые сущности (существование зависит от сильных)

2.**Связь** - ассоциация между сущностями. Может быть:

a. Один к одному (1:1)
b. Один ко многим (1:M)
c. Многие ко многим (M:M) Участие сущности в связи может быть полным или частичным.

3.**Атрибуты** - характеристики сущностей и связей. Могут быть:

a. Простыми или составными
b. Однозначными или многозначными
c. Производными или базовыми
d. Ключевыми
e. Необязательными




**Преобразование ER-модели в реляционную схему происходит по следующим правилам:**

1. Каждая сущность преобразуется в таблицу

2.Атрибуты становятся столбцами

3.Уникальный идентификатор становится первичным ключом

4.Связи «многие к одному» становятся внешними ключами

5.Создаются индексы для первичных ключей, внешних ключей и часто используемых атрибутов

6.При наличии подтипов возможны два варианта:

a. Все подтипы в одной таблице
b. Для каждого подтипа отдельная таблица

**Достоинства семантического моделирования:**

1.Позволяет выразить семантику данных

2.Упрощает проектирование базы данных

3.Обеспечивает наглядное представление структуры данных

4.Служит средством коммуникации между проектировщиком и заказчиком

**Empty list**

**3. Реляционная модель. Реляционная алгебра. Исчисление кортежей.**

Реляционная модель согласно трактовке Кристофера Дейта, состоит из трех частей: **структурной**, **целостной** и **манипуляционной**.

Структурная часть описывает из каких объектов состоит реляционная модель. Основной структурой данных в реляционной модели является нормализованные n-мерные отношения и основными понятиями структурной части реляционной модели является:

• **Тип данных** — множество значений и операций над ними (понятие такое же, как и в языках программирования);

• **Домен** можно считать уточнением типа данных и рассматривать как подмножество значений некоторого типа данных, имеющий определенный смысл. Характеризуется следующими свойствами:

– имеет уникальное имя в пределах базы данных;

– определен на некотором типе данных или на другом домене;

– может иметь логическое условие, позволяющее описать подмножество данных, доступных для данного домена;

– несет определенную смысловую нагрузку.

Домен отражает семантику, определенной предметной области и может быть несколько доменов совпадающих как подмножество, но с различным смыслом. Основное значения домена ограничивается сравнением.

• **Атрибут отношения** — это пара вида <имя атрибута, имя_домена>, при этом имена атрибутов должны быть уникальны в пределах отношения, но могут совпадать с именем домена.

• **Схема отношения** — это именованное множество упорядоченных пар <имя атрибута, имя_домена>.

• **Схема БД** — это множество именованных схем отношений.

• **Кортеж** — это множество упорядоченных пар <имя атрибута, значение_атрибута>, которое содержит одно вхождение каждого имени атрибута, принадлежащего схеме отношения.

• Отношение, определенное на множестве из n доменов (не обязательно различных), содержит две части: заголовок (схему отношения) и тело (множество из m кортежей). Значения n и m называются соответственно степенью и кардинальностью отношения.

• Непустое подмножество множества атрибутов схемы отношения будет потенциальным ключом ⇔оно будет обладать свойствами уникальности (в отношении нет двух различных кортежей с одинаковыми значениями потенциального ключа) и неизбыточности (никакое из собственных подмножеств множества потенциального ключа не обладает свойством уникальности).

• В реляционной модели по традиции один из потенциальных ключей должен быть выбран в качестве первичного ключа, а все остальные потенциальные ключи будут называться альтернативными.

• **Реляционная база данных** — это набор отношений, имена которых совпадают с именами схем отношений в схеме базы данных

**Целостностная часть** описывает ограничения специального вида, которые должны выполняться для любых отношений в любых реляционных базах данных. Это целостность сущностей и целостность внешних ключей.

**Манипуляционная часть** описывает два эквивалентных способа манипулирования реляционными данными – реляционную алгебру и реляционное исчисление

**Реляционаня алгбера**

Реляционная алгебра является основным компонентом реляционной модели, опубликованной Коддом, и состоит из восьми операторов, составляющих две группы по четыре оператора:

1. **Традиционные операции** над множествами: объединение (UNION), пересечение (INTERSECT), разность (MINUS) и декартово произведение (TIMES). Все операции модифицированы, с учетом того, что их операндами являются отношения, а не произвольные множества.
2. **Специальные реляционные операции**: ограничение (WHERE) , проекция (PROJECT), соединение (JOIN) и деление (DIVIDE BY)

Если рассматривать свойство реляционной замкнутости строго, то каждая реляционная операция должна быть определена таким образом, чтобы выдавать результат с надлежащим типом отношения (в частности, с соответствующим набором атрибутов или заголовком). Для этого вводится оператор RENAME.

**Некоторые особенности:**

• Реляционные операторы UNION, INTERSECT и MINUS требуют, чтобы отношения были совместимыми по типу, т. е. имели идентичные заголовки.

• Реляционные операторы UNION, INTERSECT, TIMES и JOIN ассоциативны и коммутативны.

• Если отношения A и B не имеют общих атрибутов, то операция соединения A JOIN B эквивалентна операции A TIMES B, т. е. в таком случае соединение вырождается в декартово произведение. Такое соединение называют естественным.

**Вывод не примитивных операторов:**

***A JOIN B = ((A TIMES (B RENAME Y AS Y1)) WHERE Y=Y1)[X, Y, Z]***

***A INTERSECT B = A MINUS (A MINUS B)***

***A DIVIDE BY B = A[X] MINUS ((A[X] TIMES B) MINUS A)[X]***

Оставшиеся реляционные операторы (объединение, вычитание, декартово произведение, ограничение, проекция) являются примитивными операторами – их нельзя выразить друг через друга

**Примеры запросов**

S(Sno: integer, Sname: string, Status: integer, City: string)

P(Pno: integer, Pname: string, Color: string, Weight: real, City: string)

SP(Sno: integer, Pno: integer, Qty: integer)

| | |
|---|---|
| Получить имена поставщиков, которые поставляют деталь под номером 2 | ( ( SP JOIN S ) WHEPE Pno = 2 ) [ Sname ] |
| Получить имена поставщиков, которые поставляют по крайней мере одну красную деталь | ( ( ( P WHERE Color = 'Красный' ) JOIN SP) [ Sno ] JOIN S ) [ Sname ]<br><br>Другая формулировка того же запроса:<br><br>( ( ( P WHERE Color = 'Красный' ) [ Pno ] JOIN SP ) JOIN S ) [ Sname ]<br><br>Этот пример подчеркивает одно важное обстоятельство: возможность сформулировать один и тот же запрос несколькими способами. |
| Получить имена поставщиков, которые поставляют все детали | ( ( SP [ Sno, Pno] DIVIDE BY P [ Pno ] JOIN S ) [ Sname ] |
| Получить номера поставщиков, поставляющих по крайней мере все те детали, которые поставляет поставщик под номером 2 | SP [ Sno, Pno ] DIVIDE BY ( SP WHEPE Sno = 2 ) [ Pno ] |
| Получить все пары номеров поставщиков, размещенных в одном городе | ( ( ( S RENAME Sno AS FirstSno ) [ FirstSno, City ] JOIN ( S RENAME Sno AS SecondSno ) [ SecondSno , City ] ) WHEPE FirstSno < SecondSno ) [ FirstSno, SecondSno ] |
| Получить имена поставщиков, которые не поставляют деталь под номером 2 | ((S[ Sno ] MINUS (SP WHEPE Pno = 2 ) [ Sno ] ) JOIN S ) [Sname] |

**Extend**

С помощью операции расширения из определенного отношения создается новое отношение, которое содержит дополнительный атрибут, значения которого получены посредством некоторых скалярных вычислений. ***EXTEND S ADD 'Supplier' AS TAG***

**Summarize**

В реляционной алгебре операция расширения позволяет выполнять "горизонтальные" вычисления в отношении отдельных строк. Оператор обобщения выполняет аналогичную функцию для "вертикальных" вычислений в отношении отдельного столбца

***SUMMARIZE SP PER SP { PNo } ADD SUM ( Qty ) AS TOTQTY***

**<u>Исчисление</u>**

Исчисление существует в двух формах: исчисление кортежей и исчисление доменов. Основное различие между ними состоит в том, что переменные исчисления кортежей являются переменными кортежей (они изменяются на отношении, а их значения являются кортежами), в то время как переменные исчисления доменов являются переменными доменов (они изменяются на доменах, а их значения являются скалярами).

Выражение исчисления кортежей содержит заключенный в скобки список целевых элементов и выражение WHERE, содержащее формулу WFF ("правильно построенную формулу"). Такая формула WFF составляется из кванторов (EXISTS и FORALL), свободных и связанных переменных, литералов, операторов сравнения, логических (булевых) операторов и скобок. Каждая свободная переменная, которая встречается в формуле WFF, должна быть также перечислена в списке целевых элементов.

| | |
|---|---|
| Получить номера поставщиков из Смоленска со статусом больше 20 | ( SX.Sno ) WHERE SX.City = 'Смоленск' AND SX.Status > 20 |
| Получить все такие пары номеров поставщиков, что два поставщика размещаются в одном городе | ( SX.Sno AS FirstSno, SY.Sno AS SecondSno ) WHERE SX.City = SY.City AND SX.Sno < SY.Sno<br><br><u>Замечание.</u><br>Спецификации «AS FirstSno» и «AS SecondSno» дают имена атрибутам результата; следовательно, такие имена недоступны для использования во фразе WHERE и потому второе сравнение во фразе WHERE «SX.Sno < SY.Sno», а не «FirstSno < SecondSno». |
| Получить имена поставщиков, которые поставляют деталь с номером 2 | SX.Sname WHERE EXISTS SPX ( SPX.Sno = SX.Sno AND SPX.Pno = 2 ) |

**Empty list**

## 4. Теория проектирования реляционной БД. Функциональные зависимости, нормальные формы

**Необходимо учитывать два фактора:**

1. Каким образом отобразить объекты предметной области в абстрактные объекты модели данных, чтобы это отображение не противоречило семантике предметной области, и было, по возможности, лучшим (эффективным, удобным и т. д.)? Часто эту проблему называют **проблемой логического проектирования** баз данных.
2. Как обеспечить эффективность выполнения запросов к базе данных? Эту проблему обычно называют **проблемой физического проектирования** баз данных.

**Нормальные формы**

Рассмотрим проблему логического проектирования. Процесс проектирования заключается в определении сущностей и отношений с дальнейшей нормализацией этих отношений с целью снижения избыточности данных. Рассматривается последовательность нормальных форм, каждая из которых накладывает все большие и большие требования к модели данных.

1) **Первая нормальная форма (1НФ/1NF)**. Отношение находится в 1НФ, если все его атрибуты являются простыми, все используемые домены должны содержать только скалярные значения, при этом нет повторяющихся кортежей.

*Пример нарушения:*
Для каждой модели надо сделать свою строку

| Фирма | Модели |
|---|---|
| BMW | M5, X5M, M1 |
| Nissan | GT-R |

2) **Вторая нормальная форма (2НФ/2NF)**. Отношение находится во 2НФ, если оно находится в 1НФ и каждый неключевой атрибут неприводимо зависит от первичного ключа, т. е. в составе потенциального ключа отсутствует подмножество атрибутов, от которых также можно вывести неключевые атрибуты.

Пример нарушения: Цена зависит от марки и модели,
а скидка только от марки!
(Декомпозируем на два отношения как слева)

| Модель | Фирма | Цена | Скидка |
|---|---|---|---|
| M5 | BMW | 5500000 | 5% |
| X5M | BMW | 6000000 | 5% |
| M1 | BMW | 2500000 | 5% |
| GT-R | Nissan | 5000000 | 10% |

| Модель | Фирма | Цена |
|---|---|---|
| M5 | BMW | 5500000 |
| X5M | BMW | 6000000 |
| M1 | BMW | 2500000 |
| GT-R | Nissan | 5000000 |

| Фирма | Скидка |
|---|---|
| BMW | 5% |
| Nissan | 10% |

3) **Третья нормальная форма (3НФ/3NF)**. Отношение находится в 3НФ, когда находится во 2НФ и каждый нетранзитивно зависит от первичного ключа.

Пример нарушения – транзитивные отношения
(Снова декомпозируем)

| Модель | Магазин | Телефон |
|---|---|---|
| BMW | Риал-авто | 87-33-98 |
| Audi | Риал-авто | 87-33-98 |
| Nissan | Некст-Авто | 94-54-12 |

4) **Нормальная форма Бойса-Кодда (НФБК)**. Отношение находится в НФБК, когда каждая нетривиальная и неприводимая слева функциональная зависимость обладает потенциальным ключом в качестве детерминанта.

5) **Четвертая нормальная форм (4НФ/4NF)**. Отношение находится в НФБК, если оно находится в НФБК и все нетривиальные многозначные зависимости фактически являются функциональными зависимостями от ее потенциальных ключей.

6) **Пятая нормальная форма (5НФ/5NF).** Отношение находится в 5НФ ⇔ каждая нетривиальная зависимость соединения определяется потенциальным ключом этого отношения.

Процесс проектирования реляционной базы данных на основе метода нормализации преследует две основные цели: избежать избыточность хранения данных и устранить аномалии обновления отношений (под этим подразумеваются определенные трудности, появляющиеся при выполнении операций обновления INSERT, DELETE, UPDATE)

**Функциональные зависимости**

Пусть R — это отношение, а X и Y – произвольные подмножества множества атрибутов отношения R. Тогда Y функционально зависимо (ФЗ) от X, что в символическом виде записывается как X → Y ⇔ ∀значение множества X связано в точности с одним значением множества Y. (Левая ФЗ часть — детерминант, правая — зависимая часть).

Очевидным способом сокращения размера множества ФЗ было бы исключение тривиальных зависимостей, т. е. таких, которые не могут не выполняться.

Множество всех ФЗ, которые задаются данным множеством ФЗ S, называется замыканием S и обозначается символом S+.

Пусть в перечисленных ниже правилах A, B и C — произвольные подмножества множества атрибутов заданной переменной-отношения R, а символическая запись AB означает {A,B}. Тогда правила вывода определяются следующим образом:

**Теорема (Правила вывода).**

Каждое и этих правил может быть непосредственно доказано на основе определения ФЗ. Более того, эти правила являются полными в том смысле, что для заданного множества ФЗ S минимальный набор ФЗ, которые подразумевают все зависимости из множества S, может быть выведен из S на основе этих правил. Они также являются исчерпывающими, поскольку никакие дополнительные ФЗ (т.е. ФЗ, которые не подразумеваются ФЗ множества S) с их помощью не могут быть выведены. Иначе говоря, эти правила могут быть использованы для получения замыкания S+.

Правила Армстронга, $R(A,B,C,D)$, $AB=\{A,B\}$:

1) Рефлексивность $B \in A \Rightarrow A \to B$
2) Дополнение $A \to B \Rightarrow AC \to BC$
3) Транзитивность $A \to B,\ B \to C \Rightarrow A \to C$
4) Самоопределение $A \to A$
5) Декомпозиция $A \to BC \Rightarrow A \to B,\ A \to C$
6) Объединение $A \to B,\ A \to C \Rightarrow A \to BC$
7) Композиция $A \to B,\ C \to D \Rightarrow AC \to BD$

**Дополнительно**

(A->B) и (C->D) следовательно (A(C–B)->BD).

**Суперключ** – переменная отношения, которая содержит в виде подмножества как минимум один потенциальный ключ

**Первичный ключ** – однозначный, неизбыточный, уже не потенциальный

Два множества ФЗ S1 и S2 **эквивалентны** ⇔ они являются покрытиями друг для друга, т. е. S1+ = S2+.

Множество ФЗ является **неприводимым** ⇔ оно обладает всеми перечисленными ниже свойствами:

1) Каждая ФЗ этого множества имеет одноэлементную правую часть.
2) Ни одна ФЗ множества не может быть устранена без изменения замыкания этого множества.
3) Ни один атрибут не может быть устранен из левой части любой ФЗ данного множества без изменения замыкания множества.

**5. Теория проектирования хранилищ данных. Основные принципы построения. ETL и ELT процессы.**

**Хранилище данных** – это система, в которой собраны данные из различных источников внутри компании и эти данные используются для поддержки принятия управленческих решений.

**Трехуровневая архитектура состоит из:**

1.Нижний уровень содержит сервер базы данных, используемый для извлечения данных из множества различных источников.

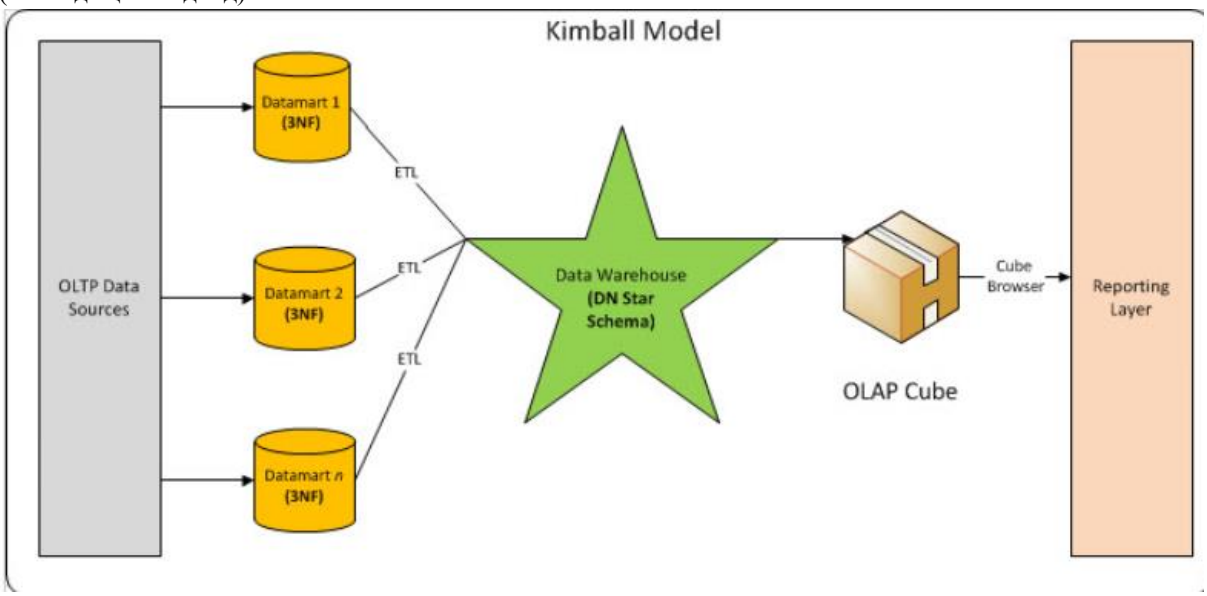
2.Средний уровень содержит сервер OLAP, который преобразует данные в структуру, подходящую для анализа и сложных запросов.

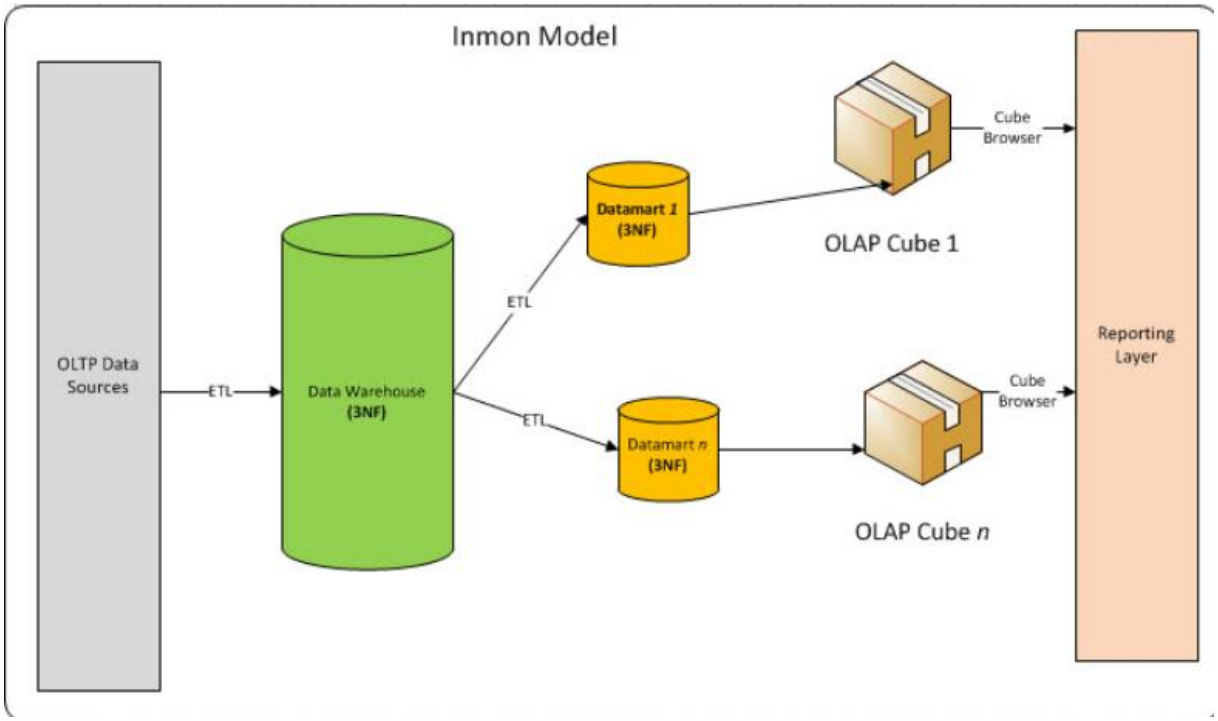
3.Верхний уровень — уровень клиента, содержащий инструменты, используемые для высокоуровневого анализа данных, создания отчетов и анализа данных.

**Есть два подхода к организации хранилища данных:**

1) Подход Ральфа Кимбалла основывается на важности витрин данных, которые являются хранилиавми данных, облегчающие отчетность и анализ. При этом организация хранилища пространственная, где существуют виртуальные объекты. Коллекция витрин данных, которые могут быть пространственно разобщенны. (восходящий подход)

2) Подход Билла Инмона основывается на том, что хранилище данных является централизованным хранилищем всех корпоративных данных. При таком подходе сначала создают нормализованную модель хранилища данных, где объектами являются физически целостными. Затем создаются витрины размерных на основе модели хранилища. Это известно как нисходящий подход к хранилищу данных. (нисходящий подход)



**ETL (Extract, Transform, Load)** сначала извлекают данные из пула источников данных. Данные хранятся во временной промежуточной базе данных. Затем выполняются операции преобразования, чтобы структурировать и преобразовать данные в подходящую форму для целевой системы хранилища данных. Затем структурированные данные загружаются в хранилище и готовы к анализу.
Инструменты ETL используются для интеграции данных, чтобы удовлетворить требованиям систем управления реляционными базами данных и/или традиционных хранилищ данных с поддержкой OLAP (online analytical processing, аналитической онлайн-обработки).
Инструменты OLAP и запросы (SQL) требуют, чтобы массивы данных структурировались и стандартизировались при помощи серии преобразований, выполняемых до того, как данные попадут в хранилище.

**ELT (Extract, Load, Transform)** данные сразу же загружаются после извлечения из исходных пулов данных. Промежуточная база данных отсутствует, что означает, что данные немедленно загружаются в единый централизованный репозиторий. Данные преобразуются в системе хранилища данных для использования с инструментами бизнес-аналитики и аналитики.

**Этапы ETL & ELT:**

1) Процесс загрузки данных из пула источников.
2) Процесс валидации (отвечает за выявление ошибок и пробелов в данных).
3) Процесс мэппинга данных.
4) Процесс агрегации данных. Этот процесс нужен из-за разности детализации данных в OLTP и OLAP системах. OLTP система может содержать несколько сумм для одного и того же набора элементов справочников, а OLAP-системы — это, по сути, полностью денормализованная таблица фактов и окружающие ее таблицы справочников.
5) Процесс выгрузки данных в целевую систему.

**6. Транзакции. Опр., свойства, уровни изоляции. Неблагоприятные эффекты из-за параллельного выполнения транзакций, способы устранения. Управление транзакциями и способы обработки ошибок.**

**Транзакция** – последовательность действий, которая выполняется как единое целое. Транзакция переводит БД из одного целостного состояния в другое.

**Свойства транзакций (ACID):**

1. **Атомарность**. Транзакция недилима, то есть выполняется либо полностью, либо не выполняется вообще.
2. **Согласованность**. БД после транзакции должна остаться в противоречивом состоянии, то есть необходимо выполнение транзакций по всем правилам, определенным в реляционной СУБД
3. **Изоляция**. Модификации одних транзакций, должны быть изолированы от модификаций выполняемых другими транзакциями параллельно. Есть несколько уровней изоляции. На каждом уровне изоляции достигается определенный компромисс между степенью распараллеливания и степенью непротиворечивости.
4. **Долговечность** (устойчивость). По завершении транзакции, ее результат должен сохраниться в системе, либо быть полностью отменен, несмотря на возможный сбой в системе.

**4 уровня изоляции по стандарту ANSI SQL. Каждый следующий включает все предыдущие**

- **0 уровень – запрет загрязнения**. Транзакции изменяют данное в режиме взаимоисключения, то есть транзакции ждут, пока другие закончат модификацию.
- **1 уровень – запрет грязного чтения**. Транзакции не могут читать данные, пока другая транзакция их модифицирует.
- **2 уровень – запрет не повторяемого чтения**. Если транзакция считывает данные, то никакая другая не может их изменить.
- **3 уровень – запрет фантомов**. Если транзакция обращается к данным, то никакая другая транзакция не сможет удалить или добавить строки, которые могут быть считаны при выполнении транзакции. В данном случае выполняется блокировка диапазона ключей, ей подвергаются некоторые строки таблицы.

**Уровни изоляции по стандарту ISO**

| Уровень | Грязное чтение | Неповторяющееся чтение | Фантомное чтение |
|---|---|---|---|
| Read Uncommitted | ДА | ДА | ДА |
| Read Committed | НЕТ | ДА | ДА |
| Repeatable Read | НЕТ | НЕТ | ДА |
| Snapshot | НЕТ | НЕТ | НЕТ |
| Serializable | НЕТ | НЕТ | НЕТ |

**Проблемы одновременного доступа:**

1. **Проблема последнего изменения.** Следующая транзакция затирает изменения, сделанные предыдущей. Решение – работа в режиме взаимоисключения.
2. **Грязное чтение**. Если изменение данных предполагает несколько операций доступа, то читатель, “вклинившись” между операциями изменения может считать данные в незавершенном виде. Решение – чтение только после полного завершения изменений.
3. **Проблема неповторяющегося чтения**. При изменении данных между повторными запросам на чтение, читающая транзакция получит несогласованные данные.
4. **Чтение фантомов**. Одна транзакция читает строки, а другая вставляет ил удаляет. В результате читающая транзакция может получить избыточные или не все строки.

**Типы транзакций по признаку определения границ**

1. **Автоматические**. По умолчанию, каждая команда выполняется как отдельная транзакция
1. **Явные транзакции**. Пользователь может объединить несколько команд в одну транзакцию, явно указав ее начало и конец. (begin, commit)
2. **Неявные транзакции.** Не существует оператора начала транзакции. Транзакция начинается с началом сеанса работы с БД. Завершается транзакция при следующих событиях:
    a. Явно выполненный оператор завершения транзакции - rollback или commit
    b. Оператор DDL
    c. Завершение сеанса.

**Управление параллельный выполнением транзакций**

1. **Пессимистичное**. Выполняется с помощью блокировок в системах с высокой степенью конкурентности
2. **Оптимистичный**. При оптимистическом подходе несколько пользователей получают в свое распоряжение копии данных. Первый завершивший редактирование сохраняет изменения, остальные же должны осуществить слияние изменений. Оптимистический алгоритм позволяет конфликту произойти, но система должна восстановиться после конфликта.

**Установка уровня изоляции**

Инструкция ***SET TRANSACTION ISOLATION LEVEL***, имеющая следующий синтаксис:

***SET TRANSACTION ISOLATION LEVEL***
***{ READ UNCOMMITTED | READ COMMITTED | REPEATABLE READ | SNAPSHOT |***
***SERIALIZABLE }***
***[ ;]***

Аргументы:

1. **READ UNCOMMITTED** – можно считывать строки, которые были изменены другими транзакциями, но еще не были зафиксированы.
2. **READ COMMITTED** – можно читать только уже зафиксированные изменения.
3. **REPEATABLE READ** – нельзя читать измененные, но не зафиксированные, нельзя изменять читаемые другой транзакцией
4. **SNAPSHOT** – данные, считанные любой инструкцией транзакции будут согласованы на уровне транзакций с версией данных, существовавших в ее начале. Транзакция распознает только те изменения, которые были зафиксированы до ее начала. Инструкции данной транзакции не видят изменений, которые внесли другие транзакции после запуска текущей транзакции. Таким образом до начала выполнения текущей транзакции, как бы был сделан снимок текущего состояния и далее все дальнейшие инструкции транзакции применяются к этому состоянию. Перед запуском транзакции , использующей уровень изоляции моментальных снимков , нужно установить параметр ALLOW_SNAPSHOT_ISOLATION в ON. Если транзакция работает с несколькими бд, этот параметр нужно установить во всех из них.
5. **SERIALIZABLE** – нельзя читать измененное и не зафиксированное, нельзя изменять читаемое, нельзя вставлять новые строки со значением ключа, которые входят в диапазон ключей, считываемых инструкции текущей транзакции, до ее завершения.

**7. Блокировки. Определение, свойства, иерархии, гранулярность и взаимоблокировки, алгоритм обнаружения взаимоблокировок.**

**Блокировка** – это механизм, с помощью которого компонент Database Engine, диспетчер блокировок, синхронизирует одновременный доступ нескольких пользователей к одному фрагменту данных. Перед доступом к фрагменту данных (разделяемому ресурсу), транзакция запрашивает его блокировку. При этом блокировка и соответствующий доступ будут предоставлены диспетчером, если запрашиваемый фрагмент данных не занят другой транзакцией. В противном случае транзакция будет добавлена в очередь. Свойства блокировки:

1. **Гранулярность**. Размер блокировки.
2. **Режим**. Тип блокировки
3. **Продолдительность**

**Иерархия блокировок** – совокупность блокировок ресурса на разных уровнях гранулярности.

Таблицы демонстрирует типы ресурсов, которые могут быть заблокированы диспетчером блокировок Database Engine, а также типы самих блокировок.

| Ресурс | Описание |
|---|---|
| RID | Идентификатор строки, используемый для блокировки одной строки в куче |
| KEY | Блокировка строки в индексе, используемая для защиты диапазонов значений ключа в сериализуемых транзакциях. |
| PAGE | 8-килобайтовая (КБ) страница в базе данных, например страница данных или индекса. |
| EXTENT | Упорядоченная группа из восьми страниц, например страниц данных или индекса. |
| HOBT | Куча или сбалансированное дерево. Блокировка, защищающая индекс или кучу страниц данных в таблице, не имеющей кластеризованного индекса. |
| TABLE | Таблица полностью, включая все данные и индексы. |
| FILE | Файл базы данных. |
| APPLICATION | Определяемый приложением ресурс. |
| METADATA | Блокировки метаданных. |
| ALLOCATION_UNIT | Единица размещения. |
| DATABASE | База данных, полностью. |

**Взаимоблокировка**

**Взаимоблокировка/deadlock/тупик** – ситуация в системе, когда одна из транзакций не может завершить свои действия, поскольку вторая транзакция заблокировала нужные ей ресурсы, а вторая в то же время ожидает освобождения ресурсов первой транзакцией. Для избежания таких ситуаций запрос на ресурс совершается с таймаутом ожидания.

**Обнаружение взаимоблокировок**

Поток диспетчера планировщик периодически производит поиск взаимоблокировок согласно следующему правилу:

Интервал между поисками по умолчанию – 5 секунд. При нахождении deadlock-ов интервал поиска снижается до 100мс. Если блокировки не найдены – интервал снова становится 5 сек, если найдены – потоки, ожидающие блокировку начинают искать их.

Для поиска владельцев взаимоблокировки монитор Идентифицирует ресурс, необходимы потоку и рекурсивно реализует поиск владельцев ресурса до тех пор, пока не обнаружит цикл.

После обнаружения виновников блокировки монитор прерывает пакет выполнения одного из потоков, участвующих во взаимоблокировке.

По умолчанию прерывается работа потока откат чьей транзакции будет менее трудозатратен.

| Режим блокировки | Описание |
|---|---|
| Совмещаемая блокировка (S) | Совмещаемая Используется для операций считывания, которые не меняют и не обновляют данные, такие как инструкция SELECT. |
| Блокировка обновления (U) | Применяется к тем ресурсам, которые могут быть обновлены. Предотвращает возникновение распространенной формы взаимоблокировки, возникающей тогда, когда несколько сеансов считывают, блокируют и затем, возможно, обновляют ресурс. |
| Монопольная блокировка (X) | Используется для операций модификации данных, таких как инструкции INSERT, UPDATE или DELETE. Гарантирует, что несколько обновлений не будет выполнено одновременно для одного ресурса. |
| Блокировка с намерением | Используется для создания иерархии блокировок. Типы намеренной блокировки: с намерением совмещаемого доступа (IS), с намерением монопольного доступа (IX), а также совмещаемая с намерением монопольного доступа (SIX). |
| Блокировка схемы | Используется во время выполнения операции, зависящей от схемы таблицы. Типы блокировки схем: блокировка изменения схемы (Sch-S) и блокировка стабильности схемы (Sch-M). |
| Блокировка массового обновления (BU) | Используется, если выполняется массовое копирование данных в таблицу и указана подсказка TABLOCK. |
| Диапазон ключей | Защищает диапазон строк, считываемый запросом при использовании уровня изоляции сериализуемой транзакции. Запрещает другим транзакциям вставлять строки, что помогает запросам сериализуемой транзакции уточнять, были ли запросы запущены повторно. |

Empty list

**8. Журнализация. Операции журнала транзакций и его логическая и физическая архитектуры. Модели восстановления. Метаданные.**

**Журнализация** – механизм, с помощью которого реализуется процесс возвращения БД в согласованное состояние в случае сбоя.

**Файлы журналов**

Файлы журналов содержат все сведения журналов, используемые для восстановления базы данных. В каждой базе данных должен быть, по меньшей мере, один файл журнала, но их может быть и больше. Для имен файлов журналов рекомендуется использовать расширение LDF. Журнал транзакций нельзя ни удалять, ни изменять, если только не известны возможные последствия. Кэш журнала управляется отдельно от
буферного кэша для страниц данных.

**Операции**

1. **Восстановление отдельных транзакций**. Если ROLLBACK или если компонент Database Engine обнаруживает ошибку, записи журнала используются для отката изменений, сделанных незавершенной транзакцией.
2. **Восстановление всех незавершенных транзакций при запуске SQL Server.** Если на сервере, где работает SQL Server сбой, мб что часть данных не будет переписана из буферного кеша в файлы, и в этих файлах мб изменения незаконченные транзакциями. Будет выполнен накат каждого записанного в журнал изменения, которое, возможно, не было переписано в файл данных и откат каждой незавершенной транзакции, найденной в журнале транзакций.
3. **Накат восстановленной базы данных, файла, файловой группы или страницы до момента сбоя.** После потери оборудования или сбоя диска, затрагивающего файлы базы данных, можно восстановить базу данных на момент, предшествующий сбою. Восстановить последнюю полную резервную копию и последнюю дифференциальную резервную копию базы данных, затем последующую серию резервных копий журнала транзакций до момента возникновения сбоя. Когда последняя резервная копия журнала будет восстановлена, тогда компонент Database Engine начнет использовать данные журнала для отката всех транзакций, которые не были завершены на момент сбоя.
4. **Поддержка репликации транзакций.** Агент чтения журнала следит за журналами транзакций всех баз данных, которые настроены на репликацию транзакций, и копирует отмеченные для репликации транзакции из журнала транзакций в базу данных распространителя.
5. **Поддержка решений с резервными серверами.** Решения резервного сервера, зеркальное отображение базы данных и доставка журналов полагаются на журнал транзакций. В сценарии доставки журналов основной сервер отправляет активный журнал транзакций основной базы данных одному или более адресатам. Каждый сервер-получатель восстанавливает журнал в свою локальную базу данных-получатель.

**Модели восстановления** – это свойство базы данных, которое управляет процессом регистрации транзакций, определяет, требуется ли для журнала транзакций резервное копирование, а также определяет, какие типы операций восстановления доступны. Есть три модели восстановления:

**Full**.

a. Записываются все транзакции полностью, сохраняются до резервного копирования.
b. Позволяет восстановить данные до точки сбоя (при наличии копии журнала).
c. Поддерживает восстановление отдельных страниц данных.
d. Требует регулярного создания резервных копий журналов.

**Bulk logged**

a. Частично регистрирует массовые операции (импорт, создание индексов), что повышает производительность и экономит место.
b. Используется как временная альтернатива FULL для масштабных задач.
c. Журнал транзакций сохраняется до резервного копирования, но восстановление на конкретный момент времени невозможно.
d. Риск потерь данных выше, чем в FULL

**Simple**

a. Журнал транзакций автоматически очищается, резервные копии журнала не создаются.
b. Можно восстановить данные только до последнего резервного копирования.
c. Подходит для сценариев, где потеря данных допустима.
d. Для снижения риска потерь рекомендуется чаще делать полное или разностное резервное копирование.

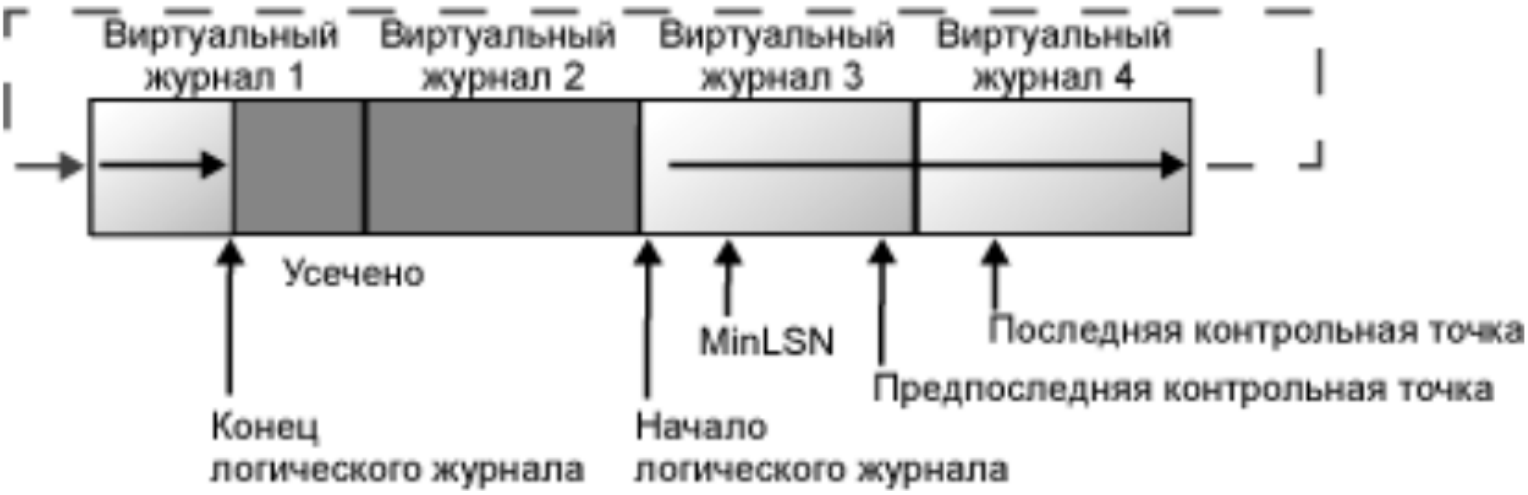
**Логическая архитектура журнала транзакций**

На логическом уровне журнал транзакций состоит из последовательности записей. Каждая запись содержит:

1. Log Sequence Number: регистрационный номер транзакции (LSN). Каждая новая запись добавляется в логический конец журнала с номером LSN, который больше номера LSN предыдущей записи.
2. Prev LSN: обратный указатель, который предназначен для ускорения отката транзакции.
3. Transaction ID number: идентификатор транзакции.
4. Type: тип записи Log-файла.
5. Other information: Прочая информация.

**Физическая архитектура журнала**

На физическом уровне журнал транзакций состоит из одного или нескольких физических файлов. Каждый физический файл журнала разбивается на несколько виртуальных файлов журнала (VLF). VLF не имеет фиксированного размера. Не существует также и определенного числа VLF, приходящихся на один физический файл журнала.



## 9. Безопасность и аудит. Ключевые понятия и участники системы безопасности. Модели управления доступом.

**Безопасность** делится на внутреннюю (ошибки и несанкционированные действия персонала, ошибки проектирования, системные ошибки) и внешнюю (умышленные действия внешних лиц с целью искажения/кражи информации, вирусы сбои, отказы в аппаратуре)

**Аудит** – процедура трассировки потоков данных в бд с целью проверки безопасности, обнаружения ошибок.

Участники системы безопасности или принципалы – это сущности, которые могут запрашивать ресурсы SQL Server. Принципалы могут быть иерархически упорядочены.

**Область влияния принципала зависит:**

1. от области его определения: Windows, SQL Server, база данных,
2. от того, коллективный это участник или индивидуальный. Имя входа Windows является примером индивидуального (неделимого) участника, а группа Windows — коллективного.

Участники уровня Windows – это a) Имя входа домена Windows, b) Локальное имя входа Windows.

Участники уровня SQL Server – это a) Имя входа SQL Server, b) Роль сервера.

Участники уровня базы данных – это a) Пользователь базы данных, b) Роль базы данных, c) Роль приложения.

Защищаемые объекты – это ресурсы, доступ к которым регулируется системой авторизации. Некоторые защищаемые объекты могут храниться внутри других, создавая иерархии «областей», которые сами могут защищаться. К областям защищаемых объектов относятся (a) сервер, (b) база данных и (c) схема.

У субъекта системы есть только один путь получения доступа к объектам – иметь назначенные непосредственно или опосредовано разрешения. При непосредственном управлении разрешениями они назначаются субъекту явно, а при опосредованном разрешения назначаются через членство в группах, ролях или наследуются от объектов, лежащих выше по цепочке иерархии. Управление разрешениями производится путем выполнения инструкций языка DCL (Data Control Language): **GRANT** (разрешить), **DENY** (запретить) и **REVOKE** (отменить).

**Примеры управления именами входа Postgres**

```
– Создание пользователя
CREATE ROLE carol WITH LOGIN PASSWORD
'Th1sI$|\/|yP@ssw0rd';
–Ассоциация пользователя с серверной ролью
GRANT securityadmin TO ted;
– В postgres нет фиксированных ролей, как в SQL Server, но мы
можем пользовательской роли назначить нужные привилегии,
например:
CREATE ROLE securityadmin;
GRANT CREATE ROLE, CREATEROLE, CREATEDB TO
securityadmin;
GRANT securityadmin TO ted;
```

**Управление учетными записями бд**

```
– Создание роли
CREATE ROLE veronica WITH LOGIN PASSWORD 'PalmTree1';
– Назначение административных привилегий на уровне сервера
ALTER ROLE veronica WITH SUPERUSER;
– Создание пользователя бд
\c TestDB – Подключаемся к базе данных TestDB
CREATE ROLE hr_reportspecialist NOLOGIN;
GRANT hr_reportspecialist TO veronica;
– Создание роли и назначение привилегий
CREATE ROLE hr_reportspecialist;
GRANT ALL PRIVILEGES ON SCHEMA public TO
hr_reportspecialist;
GRANT hr_reportspecialist TO veronica;
```

**Управление разрешениями**

```
GRANT { ALL [ PRIVILEGES ] | список_разрешений }
ON объект_или_объекты
TO { роль | PUBLIC }
[ WITH GRANT OPTION ];
```

**Параметры:**

1. ALL [ PRIVILEGES ] — Предоставляет все доступные привилегии на указанный объект.
2. список_разрешений — Указывается конкретный набор привилегий, таких как SELECT, INSERT, UPDATE, DELETE, и т.д.
3. объект_или_объекты — Таблицы, представления, последовательности, функции или схемы, к которым предоставляются привилегии.
4. TO { роль | PUBLIC } — Указывает, кому предоставляются привилегии. Это может быть конкретная роль или ключевое слово PUBLIC (для всех пользователей).
5. WITH GRANT OPTION — Позволяет получателю передавать привилегии другим пользователям.
6. AS принципал — В PostgreSQL нет прямого аналога этого параметра из SQL Server. Привилегии назначаются от имени владельца объекта. Принципалом в SQL Server может быть пользователь базы данных, роль базы данных, роль приложения.

**Модели управления доступом:**

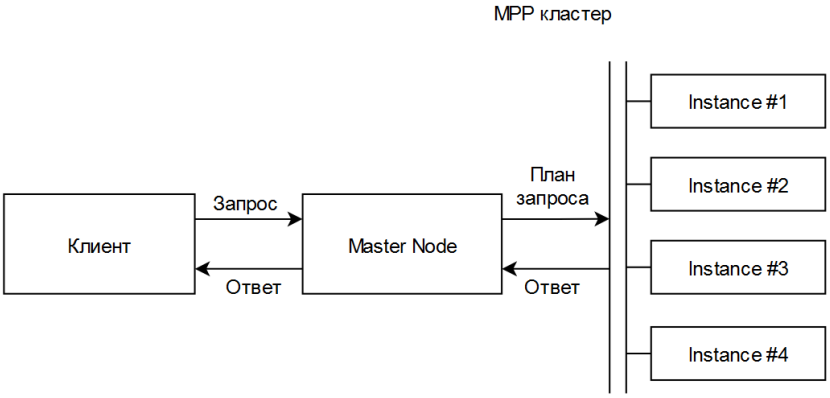
1) MAC (Mandatory Access Control) – обязательное управление доступом. Вводится модель секретности. Как в госструктурах – открытый доступ, конфиденциально, секретно, совершенно секретно.
2) DAC (Discretionary Access Control) – по усмотрению. Владелец ресурса определяет доступ конкретного пользователя к нему.
3) RBAC (Role-Based Access Control) – администратор назначает роли, от ролей зависят доступы к ресурсам.
4) ABAC (Attribute-Based Access Control) – производятся вычисления на основе атрибутов (имя, роль, пропуск…) по определенным правилам. Подходит, если сложная иерархия.

Empty list

## 10. MPP системы. Распределенное и колоночное хранение. Распределенные вычисления, модель MapReduce. Обеспечение отказоуст.

### MPP (Massive Parallel Processing) системы

**Идея**: фрагментировать данные и предоставить обработку каждого фрагмента своей машине (свои CPU и память).




**Преимущества:**

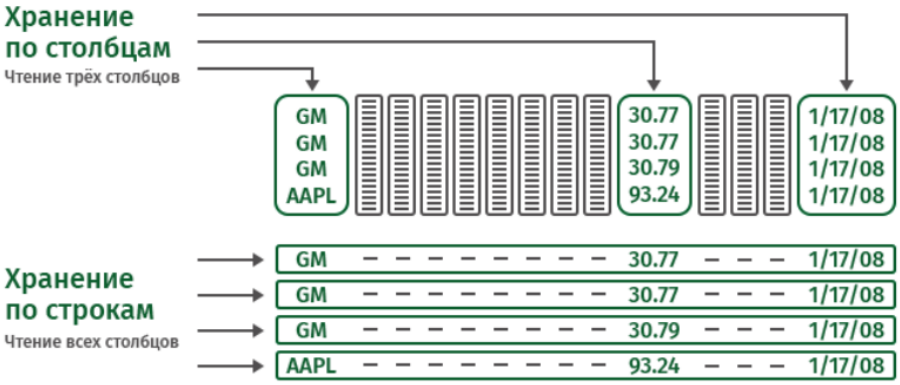
1. Относительное быстродействие при обработке больших данных
2. Простое горизонтальное масштабирование
3. Отказоустойчивость за счет распределенной архитектуры – каждый узел независим от других

**Недостатки:**

1. Неэффективность при большом количестве простых запросов
2. Неэффективность из-за неравномерного распределения фрагментов по инстансам
3. Горизонтальное масштабирование требует большого кол-ва аппаратных ресурсов

### Колоночное и строковое хранение (пример такой БД: Vertica)

В колоночных БД данные на диске сгруппированы по столбцам, то есть все значения конкретного атрибута лежат рядом.



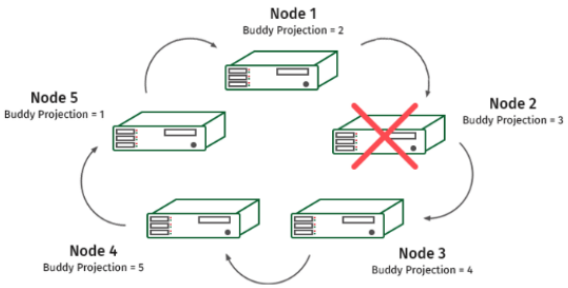

| Критерий | Ориентация на строки | Ориентация на столбцы |
|---|---|---|
| Применение | OLTP-приложения (обработка транзакций) | OLAP-приложения (аналитическая нагрузка) |
| Эффективность записи | Высокая, операции вставки и удаления данных выполняются просто и быстро | Низкая, т.к. в денормализованной схеме приходится обращаться к нескольким таблицам |
| Степень сжатия | Низкая, хранение данных занимает больше места на диске | Высокая, благодаря возможности применять наиболее подходящие алгоритмы для отдельных столбцов, где хранятся данные одного типа |
| Эффективность чтения | Низкая из-за считывания ненужных данных | Высокая, благодаря прямому обращению к нужным столбцам |
| Примеры | MySQL, PostgreSQL | ClickHouse, Cassandra, Snowflake, Amazon Redshift, BigQuery, HBase |

### Отказоустойчивость

Одним из методов обеспечения отказоустойчивость в распределенной системе является введение “Buddy Projections”. Принцип заключается в том, чтобы хранить в узлах не только свои данные, но и копии данных других узлов. Если на узле M хранится копия проекции данных узла T, то в случае отказа узла T, его работа ляжет на плечи узла M.

**Пример**

На пример с картинки в нагрузка с узла 2 перейдет на первый узел, так как он хранит buddy projection второго узла.



**<u>Распределенные вычисления и MapReduce</u>**

MapReduce – это технология распределенных вычислений, которая используется для обработки огромного объема данных. Суть MapReduce:

1. Разделить входной массив данных на фрагменты
2. Обработать каждый фрагмент независимо
3. Слить результаты обработки фрагментов

**Модель MapReduce**

1. Ввод/вывод данных в формате K-V
2. Функция MAP отображает пару K-V на некоторое промежуточное множество пар K-V и REDUCE, отображающая некоторое множество значений, имеющих общий ключ, на меньшее множество значений
3. Стадия Shuffle & sort необходима для сортировки ввода в REDUCER по ключу K, чтобы пары с одинаковыми ключами шли в один REDUCER.

**Стоит использовать, если:**

1. Данные компонуемые, то есть предполагают независимую обработку и слив результатов вместе
2. Большие объемы данных

Не стоит использовать в случае, если отрицание одного из двух пунктов выше – истинно.

## 11. In-memory базы данных. Преимущества и недостатки. Примеры использования.

### In-memory базы данных

In-memory база данных целиком расположена в оперативной памяти. Выигрыш в скорости доступа к данным, проигрыш в энергозависимости (обесточивание сервера ведет к потере данных). Снизить риск потери данных можно с помощью следующих методов:

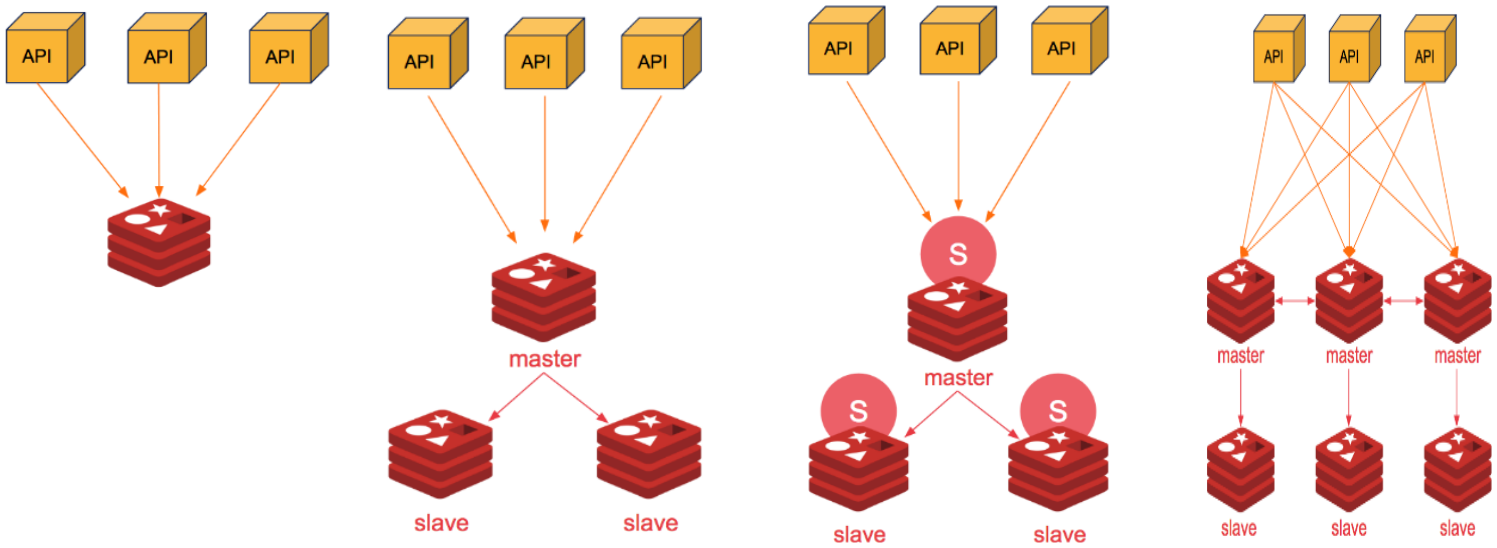
1. Снапшот данных. Периодическое сохранение данных на диск. Проблема состоит в потере самых актуальных данных в случае неполадки.
2. Логирование транзакций на диск.
3. Использование гибридных аппаратных решений типа NVRAM, которые сохраняют свое состояние при потере питания. Данный метод показывает время доступа к данным хуже, ведь это уже не чистый RAM

**Redis – in-memory K-V хранилище**

Типы данных: строки, списки, множества, упорядоченные множества, хеш-таблицы

**Топология Redis**

1. `Один Redis instance. Ограничен ресурсами хоста, на котором запущен. Не является отказоустойчивым: хост отказал → Redis упал`
2. Master-slave репликация. Запросы приходят на master node, после чего копируются по salve-ам; Slave-ы имеют формат READ ONLY; Операции ограничены ресурсами хоста master node
3. Redis Sentinel. Аналог Master-slave, но добавляются Sentinel узлы, которые выбирают новый master узел в случае отказа предыдущего
4. Redis cluster. Несколько master узлов, у каждого свой slave-ы.




**Примеры использования:**

1. Кэширование запросов
2. Управление сессиями (аутентификация)
3. Статистика в реальном режиме
4. Обмен сообщениями

**Преимущества in-memory:**

1. Быстрота выполнения операций
2. Повышение пропускной способности highload систем

**Недостатки in-memory:**

1. Однопоточность и возможность утилизации только одного ядра CPU. При проектировании систем с in-memory хранилищами стоит закладывать потенциальное масштабирование на много инстансов.

Энергозависимость

Empty list

## 12. Инструкции языка описания данных, инструкции языка обработки данных, инструкции безопасности, инструкции управления транзакциями.

### Язык описания данных DDL (Data Definition language)

Назначение: описание структуры БД. Создание объекта ***create***, изменение объекта ***alter***, удаление объекта ***drop***.

**Способы представления данных**

1. Таблица
2. Временная таблица. Существует на протяжении сессии БД (например, пока открыто окно запросов). Создание аналогично обычной таблице, но в начале имени надо добавить решетку. ***create table #temp_table;***
3. Представление (view). Виртуальные таблицы, содержащие запросы, динамически извлекающие данные из БД. Представление физически не хранит данные и ничего не весит.
4. Табличные переменныеИндексированное представление
5.

Создание

***CREATE TABLE IF NOT EXISTS tab_name (***
***attrname type[,]***
***);***

Изменение

***ALTER TABLE Employees ALTER COLUMN ID int NOT NULL;***

Удаление

***DROP TABLE tab_name;***

### Язык обработки данных DML (Data Manipulation Language)

Назначение: манипуляции над данными. Вставка ***insert***, обновление ***update***, выборка ***select***, удаление ***delete***

***SELECT column_list FROM table_name [WHERE условие] [GROUP BY условие] [HAVING условие] [ORDER BY условие]***

### Инструкции безопасности DCL (Data Control Language)

SQL сервер обеспечивает защиту данных от неправомерного доступа и фальсификации данных. Основные функции:

1. Аутентификация. Проверка подлинности
2. Авторизация. Предоставление некоторых прав.

Основные инструкции:

1. GRANT – выдать права на некоторые операции с объектом
2. REVOKE – отозвать права
3. DENY – запретить
4. EXECUTE AS – контекст выполнения сеанса переключается на заданное имя пользователя
5. REVERT – переключить контекст выполнения на последнего пользователя, который вызвал EXECUTE AS

### Инструкции управления транзакциями TCL (Transaction Control Language)

1. BEGIN DISTRIBUTED TRANSACTION — запускает распределенную транзакцию, управляемую координатором распределенных транзакций.
2. BEGIN TRANSACTION — отмечает начальную точку явной локальной транзакции.
3. COMMIT TRANSACTION — отмечает успешное завершение явной или неявной транзакции.
4. COMMIT WORK — действует так же, как и инструкция COMMIT TRANSACTION.
5. ROLLBACK TRANSACTION — откатывает явные или неявные транзакции до начала или до точки сохранения транзакции.
6. ROLLBACK WORK — действует так же, как и инструкция ROLLBACK TRANSACTION.
7. SAVE TRANSACTION — устанавливает точку сохранения внутри транзакции.

## 13. Объекты базы данных: функции, процедуры, триггеры и курсоры.

### Функции и процедуры SQL

Функции бывают детерминированные (при одном и том же входном данном одно и то же выходное) и недетерминированными. Пример недетерминированной функции – getdate(), которая возвращает текущую дату на SQL сервере.

**В SQL выделяют следующие типы функций:**

Скалярная функция

Общий синтаксис

```
CREATE FUNCTION [ имя-схемы. ] имя-функции ( [ список-объявлений-параметров ] )
RETURNS скалярный-тип-данных
[ WITH список-опций-функций ]
[ AS ]
BEGIN
    тело-функции
    RETURN скалярное-выражение
END [ ; ]
```

Пример:

***CREATE FUNCTION getSalary(int)***
***RETURNS int AS $$***
***SELECT max(Salary) AS result FROM foo WHERE id = $1; $$ LANGUAGE SQL;***

Подставляемая табличная функция

Общий синтаксис

```
CREATE FUNCTION [ имя-схемы. ] имя-функции ( [ список-объявлений-параметров ] )
RETURNS TABLE
[ WITH список-опций-функций ]
[ AS ]
    RETURN [ ( ] выражение-выборки [ ) ]
END [ ; ]
```

Пример

***CREATE FUNCTION getfoo(int)***
***RETURNS foo AS***
***$$ RETURN QUERY SELECT * FROM foo WHERE fooid = $1; $$ LANGUAGE SQL;***

Многооператорная табличная функция

Общий синтаксис

```
CREATE FUNCTION [ имя-схемы. ] имя-функции ( [ список-объявлений-параметров ] )
RETURNS @имя-возвращаемой-переменной TABLE определение-таблицы
[ WITH список-опций-функций ]
[ AS ]
BEGIN
    тело функции
    RETURN
END [ ; ]
```

Пример

***CREATE FUNCTION sum_n_product_with_tab (x int)***
***RETURNS TABLE(sum int, product int) AS***
***$$ SELECT x + tab.y, $1 * tab.y FROM tab; $$ LANGUAGE SQL;***

**Некоторые отличия функций и процедур в SQL**

1. В процедуре можно использовать все ключевые слова языка DML, в функции только SELECT
2. Процедура вызывается с помощью CALL, функция с помощью SELECT

### Триггер

**Триггер** - это хранимая процедура особого типа, которая выполняет одну или несколько инструкций в ответ на событие. Выделяют DML и DDL триггеры.

Триггер DDL может активироваться, если выполняется такая инструкция, как ALTER SERVER CONFIGURATION, или если происходит удаление таблицы с использованием команды DROP TABLE.

```
CREATE EVENT TRIGGER имя
ON событие
[ WHEN переменная_фильтра
  IN (значение_фильтра
  [, ... ])
[ AND ... ] ]
EXECUTE PROCEDURE имя_функции()
```

Пример триггера:

```
CREATE OR REPLACE FUNCTION
abort_any_command()
RETURNS event_trigger
LANGUAGE plpgsql AS
$$
  BEGIN RAISE EXCEPTION 'команда %
        отключена', tg_tag;
END;
$$;

CREATE EVENT TRIGGER abort_ddl
ON ddl_command_start

EXECUTE PROCEDURE abort_any_command();
```

**DML триггеры**

1. INSTEAD OF. Триггер выполняет подменяющее действие
2. BEFORE/AFTER. Триггер выполняется до или после действия, на которое определен триггер.

Пример триггера AFTER. При добавлении в базу нового клиента ему будет выдана подписка на 10 дней

```
CREATE OR REPLACE FUNCTION create_subscription_for_new_client()
RETURNS TRIGGER AS $$
DECLARE
        new_subscription_id UUID;
BEGIN
        new_subscription_id := uuid_generate_v4();
        INSERT INTO subscription (id, expires_at)
        VALUES (new_subscription_id, CURRENT_TIMESTAMP + INTERVAL '10 days');
        UPDATE client
        SET id_subscription = new_subscription_id
        WHERE id = NEW.id;

        RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER sub_for_new_clients
AFTER INSERT ON client
FOR EACH ROW
EXECUTE FUNCTION create_subscription_for_new_client();
```

Пример триггера INSTEAD OF. При попытке вставки в представление вставка будет происходить в реальные таблицы

```
CREATE OR REPLACE VIEW owner_subscription_view AS
SELECT
  c.id AS owner_id, s.id AS subscription_id, s.expires_at
FROM client c
JOIN subscription s ON c.id_subscription = s.id;

CREATE OR REPLACE FUNCTION handle_owner_subscription_update()
RETURNS TRIGGER AS $$
BEGIN
  UPDATE subscription
  SET expires_at = NEW.expires_at
  WHERE id = NEW.subscription_id;
  RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER instead_of_owner_subscription_update
INSTEAD OF UPDATE ON owner_subscription_view
FOR EACH ROW
EXECUTE FUNCTION handle_owner_subscription_update();
```

**Курсор**

Операции в реляционной базе данных выполняются над множеством строк. Например, набор строк, возвращаемый инструкцией SELECT, содержит все строки, которые удовлетворяют условиям, указанным в предложении WHERE инструкции.

Курсор является некоторым аналогом итератора в применении к обработке данных в БД.

**Курсор позволяет:**

1. Позиционироваться на определенных строках результирующего набора
2. Получать одну или несколько строк от текущей позиции в результирующем наборе;
3. Подерживать изменение данных в строках в текущей позиции результирующего набора;
4. Поддерживать разные уровни видимости изменений, сделанных другими пользователями для данных, представленных в результирующем наборе;

**Классификация курсоров:**

1. По типу
    a. Static. Создается временная копия данных для использования курсором
    b. Dynamic.
    c. Keyset. Порядок строк в курсоре неизменен после его открытия
    d. Fast forward
2. По обласи видимости
    a. Локальный. В рамках пакета, процедуры или триггера
    b. Глобальный. В рамках сессии
3. По типу перемещения по курсору
    a. Forward only. От первой записи к последней
    b. Scroll. Вперед и назад
4. По типу распараллеливания
    a. Read only. Содержимое курсора можно только читать
    b. Scroll locks. При редактировании записи никто другой не может вносить в нее изменения
    c. Optimistic. Отсутствие блокировок. Даже при редактировании данных другие пользователи могут к ним обращаться.

**Для перемещения по курсору испольуется ключевое слово FETCH с параметрами:**

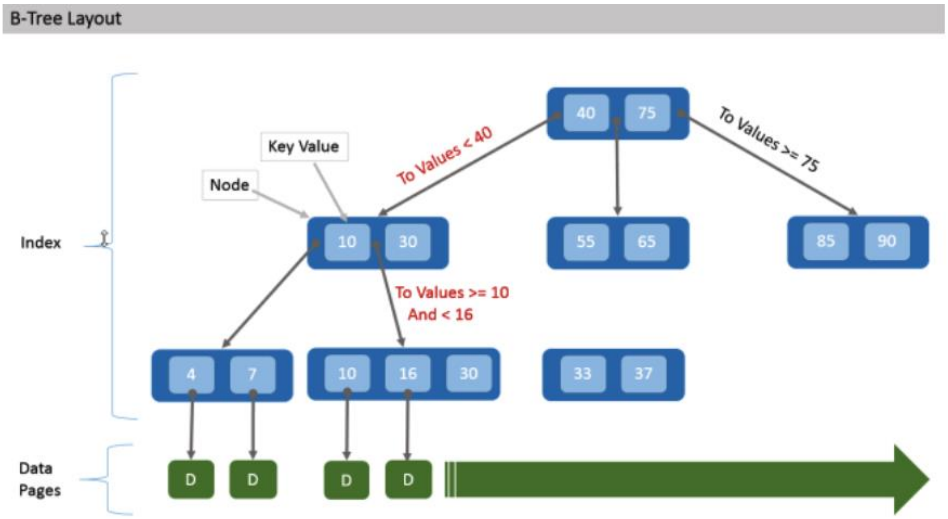
1. NEXT (по умолчанию). На следующую строку
2. PRIOR. На предыдущую строку.
3. FIRST. На первую строку.
4. LAST. На последнюю строку.
5.

Пример использования курсора (Инкрементируем счстчик, если нашли имя)

```
create procedure updateNameStat(name)
begin
  declare cnt int default 0;
  declare exit handler for sqlstate not found begin
        insert into Stats (Name, Cnt) values (name, cnt);
  end;
  declare SCursor cursor
        select FirstName from Students;
  open SCursor;
  loop
        declare sname varchar(20);
        fetch SCursor into sname;
        if name = sname then set cnt = cnt + 1; end if;
  end loop;
end;
```

## 14. Оптимизация запроса: индексы, партицирование, сегментирование.

**Индекс** – объект БД, позволяющий ускорить поиск данных в БД. Недостатком использования является дополнительное место на диске и замедление операций вставки и удаления. Индексы в БД реализованы с помощью структуры данных B-дерево:




Создание индекса:

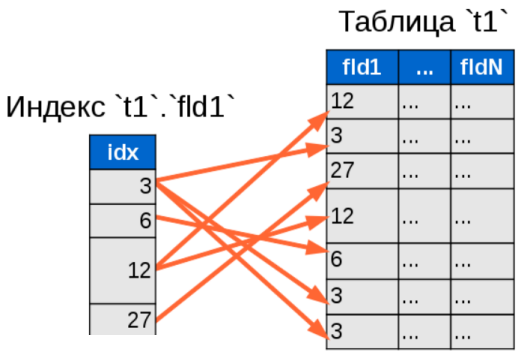
CREATE [ UNIQUE ] [ CLUSTERED | NONCLUSTERED ]
INDEX имя-индекса
ON имя-таблицы-или-представления
( список-столбцов )
[ INCLUDE (список-столбцов) ]
[WITH список-опций]
[ ON файловая-группа ]

```
— Создание индекса отдельно от таблицы
CREATE UNIQUE NONCLUSTERED INDEX [IX_Address]
ON [Person].[Address]([AddressLine1] ASC,
                      [AddressLine2] ASC,
                      [City] ASC,
                      [StateProvinceID] ASC,
                      [PostalCode] ASC)

— Создание индекса вместе с ограничением
CREATE TABLE [Person].[Address](
  [AddressID] int IDENTITY(1,1) NOT NULL,
  ...
  CONSTRAINT [PK_Address_AddressID] PRIMARY KEY
      CLUSTERED([AddressID] ASC) ON [PRIMARY]
) ON [PRIMARY]
```

**Индексы бывают кластерные и некластерные (обычные):**

Обычный индекс подобен табличке, строки которой указывают на элементы целевой таблицы. Такая таблица мало весит, кешируется в RAM, а ключи в ней отсортированы, что позволяет применять к ее элементам Бинарный

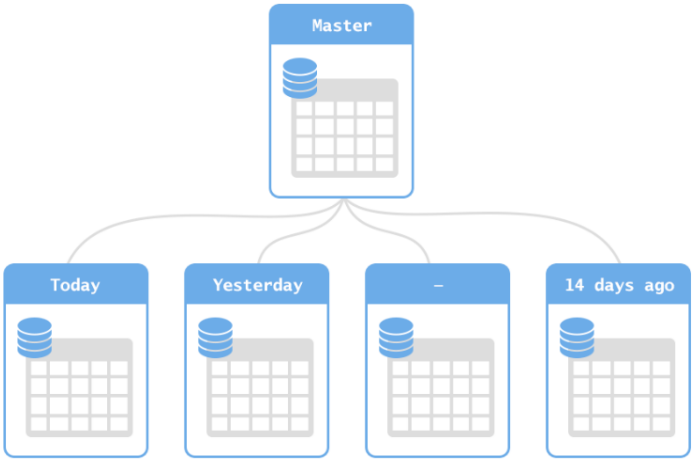



Кластерные индексы представляют собой древовидную структуру, и находятся вместе с данными. Если обычный индекс указывает на точное положение данного в таблице, то кластерный индекс содержит информацию о промежутке, в котором может находиться искомое данное.

**Партиционирование**

**Партиционирование** – разделение больших по кол-ву строк таблиц на несколько по некоторому признаку.

***CREATE TABLE measurement (***
***city_id int not null,***
***logdate date not null,***
***peaktemp int,***
***unitsales int***
***) PARTITION BY RANGE (logdate);***



**Идеи партиционирования с преимуществами и недостатками**

1. **По дате**. (+) Равномерность распределения, (--) Поиск по ID требует просмотра всех партиций
2. **По первой букве имени**. (+) Не нужно создавать новые партиции со временем. (--) Неравномерность распределения, поиск по ID требует просмотра всех партиций.
3. **По идентификатору.** (+) Оптимально при поиске по ID. (--) Обновление партиций со временем

**<u>Сегментирование</u>**

**Сегментирование** – процесс распределения проекций данных по кластерам. Процесс сегментирования подобен распределению данных в хеш-таблице. Разработчику необходимо выбрать список полей и хеш функцию, которая позволит распределить данные по кластерам равномерно.

Создадим простую таблицу:

```
CREATE TABLE test_order (
  n integer primary key,
  attr1 text,
  attr2 text,
  dt timestamp
)
Distributed by (n)
Partition by dt;
```

Создадим две сообщённые проекции:

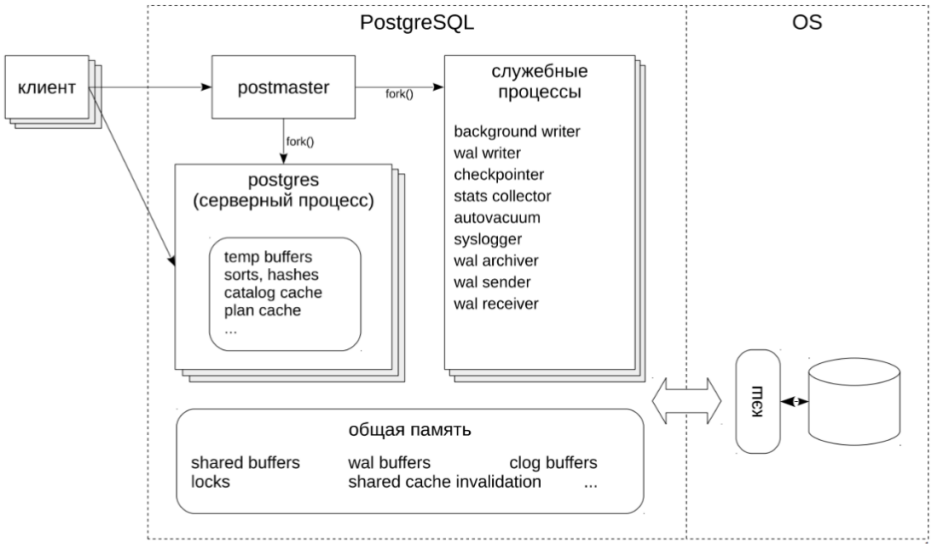
```
CREATE PROJECTION p_test_b0 AS
SELECT n FROM test_order BY n
SEGMENTED BY hash(n) ALL NODES;

CREATE PROJECTION p_test_b1 AS
SELECT n, attr1 FROM test_order BY n
SEGMENTED BY hash(n) ALL NODES offset 1;
```

**15. План выполнения запроса. Этапы выполнения запроса.**

**Структура PostgreSQL**

PostgreSQL имеет клиент-серверную архитектуру:




**Стадии, обработки клиентского запроса:**

1. **Соединение.** Прикладная программа устанавливает подключение к серверу, передает запрос и ждет ответ.
2. **Разбор запроса.** Синтаксическая проверка запроса и создание дерева запроса. Дерево имеет узлы двух типов:
   a. Атомы (ключевые слова SQL, имена, константы, скобки, операторы, …)
   b. Синтаксические категории (имена семейств, представляющие части запроса, они заключаются в фигурные скобки <SFW>, <Condition>).
3. **Применение системы правил.** Преобразование дерева разбора предыдущего пункта на основе некоторых правил.
4. **Создание плана запроса**. Планировщик принимает дерево запроса и выбирает план запроса, проанализировав все возможные варианты получения результата.
5. **Результат**. Исполнитель на основе плана получает строки результата.

**План запроса**

План запроса показывает, как будет производиться сканирование таблиц (последовательно, по индексу, …) и какой алгоритм соединения таблиц будет применен, если таковых несколько. У запроса есть ожидаемая стоимость, единицей которой обычно является стоимость обращения к странице на диске. Есть два типа стоимость:

1. До первой строки результата. Например, в запросах с EXISTS планировщик будет оптимизировать именно ее, так как обнаружения одной строки достаточно для ответа на EXISTS.
2. Общая стоимость выдачи всех строк.

*Получение плана в Postgres: EXPLAIN [ params ] operator*

**Устройство плана запроса**

План запроса представляется деревом узлов плана.

**Пример №1**

*explain select * from tab;*

*→ Seq scan on tab (costs=0.00..458.00 rows=1000, width=1000)*

Параметры в скобках по очереди: стоимость запуска, общая стоимость, ожидаемое число строк, которые вернет узел, ожидаемый размер строк в байтах.

Такая информация выводится для каждого узла дерева плана.

**Пример №2**

```
explain
SELECT Sname
FROM dbSPJ.S s JOIN dbSPJ.SPJ spj
               ON s.Sno = spj.Sno
WHERE Qty > 500;
```

```
QUERY PLAN
Hash Join  (cost=23.05..120.34 rows=1789 width=58)
  Hash Cond: (spj.sno = s.sno)
  -> Seq Scan on spj  (cost=0.00..33.12 rows=617 width=4)
     Filter: (qty > 500)
  -> Hash  (cost=15.80..15.80 rows=580 width=62)
     -> Seq Scan on s  (cost=0.00..15.80 rows=580 width=6
```

Empty list

## 16. Гибкие методологии проектирования. Data Vault, Anchor Modeling.

**Мотивация**: данные в корпорациях хранятся разрозненно – у каждого отдела своя база. Такой подход хорош при разделении зон ответственности (HR отдел не имеет доступа к данным бухгалтерии), но усложняет процесс сквозной аналитики данных компании в целом.

**Решение**: Data Warehouse (DWH) – это хранилище, которое собирает в себе данные компании из разных систем. Признаки DWH:

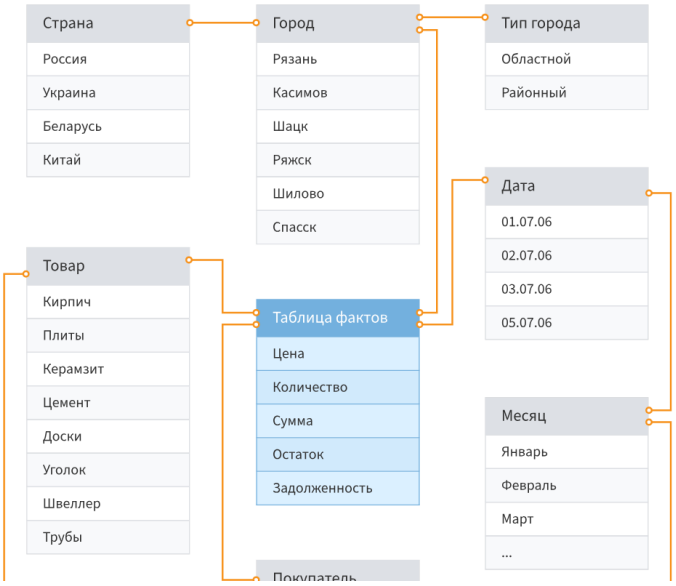
1. Подлежащие анализу данные хранятся в одном месте – хранилище
2. Данные в хранилище можно добавлять, удалять, очищать, выгружать
3. С использованием BI инструментов (Grafana) у пользователей есть возможность выявлять закономерности и собирать сквозную статистику.

**Модели хранения данных для DWH.**

Снежинка

Данная модель состоит из одной таблицы фактов и нескольких таблиц измерений.

Центральная таблица фактов содержит ключевые показатели, ассоциированные с измерениями бизнес-процессов. Таблица фактов связывается с таблицами измерений через внешние ключи.




Преимущества:

1. Структура соотносится с моделью данных
2. Можно настроить проверку консистентности данных

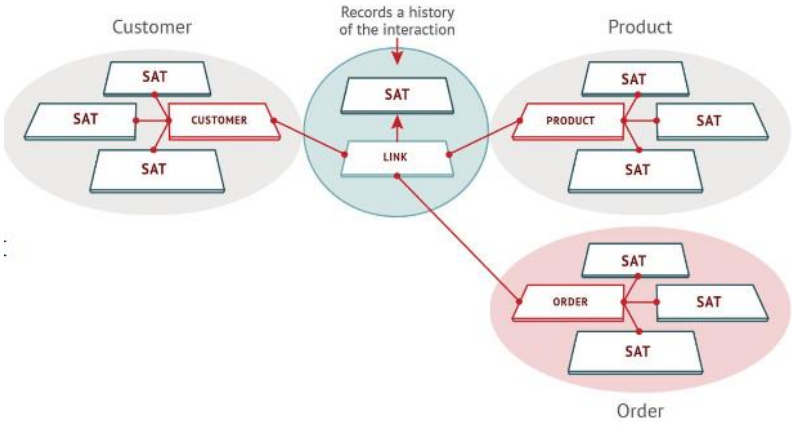
Недостатки:

1. Не подходит для проектов с обширной или быстро меняющейся архитектурой
2. Снижается производительность запросов
3. При добавлении данных требуется множество согласований

Data Vault

Данный тип хранилища состоит из трех сущностей:

1. **Хаб (hub)** – бизнес-сущность, например клиент, продукт, заказ. Хаб-таблица содержит несколько полей: натуральные ключи сущности, суррогатный ключ, ссылка на источник записи и время добавления записи
2. **Линк (связь, link)** – представление отношений между Хабами. Линки строятся в виде таблиц «многие ко многим» и содержат ссылки на суррогатные ключи связанных Хабов.
3. **Сателлит (satellite)** – изменяемые атрибуты сущностей, контекстные данные для хаб-таблицы, связанные с Хабами и Связями по принципу «один ко многим».




Преимущества:

1. По анализу хабов и связей можно составить верхнеуровневый отчет
2. Низкая связность данных

Недостатки:

1. Большое кол-во таблиц и Join операций замедляет выполнение
2. Сложность выстраивания процесса проверки данных

Якроная модель (Anchor modeling)

Данный тип хранилища состоит из четырёх сущностей:

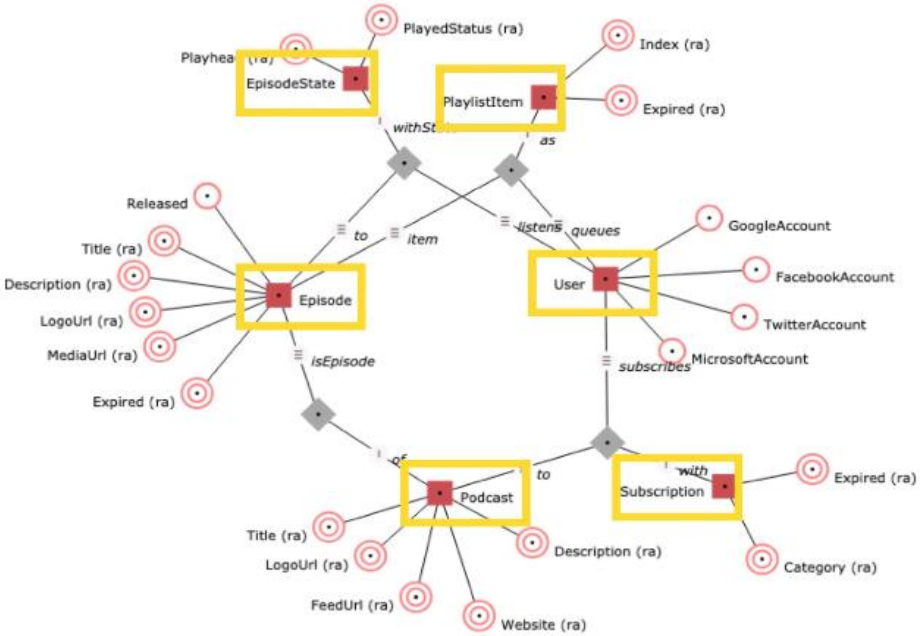
1. **Якорь** – представляет собой сущность или событие, содержит суррогатные ключи, ссылку на источник и время добавления записи
2. **Атрибут** – используется для моделирования свойств и характеристик якорей, содержит суррогатный ключ якоря, значение атрибута, ссылку на источник записи и время добавления записи
3. **Связь** – моделирует отношения между якорями
4. **Узел** – используется для моделирования общих свойств (состояния)

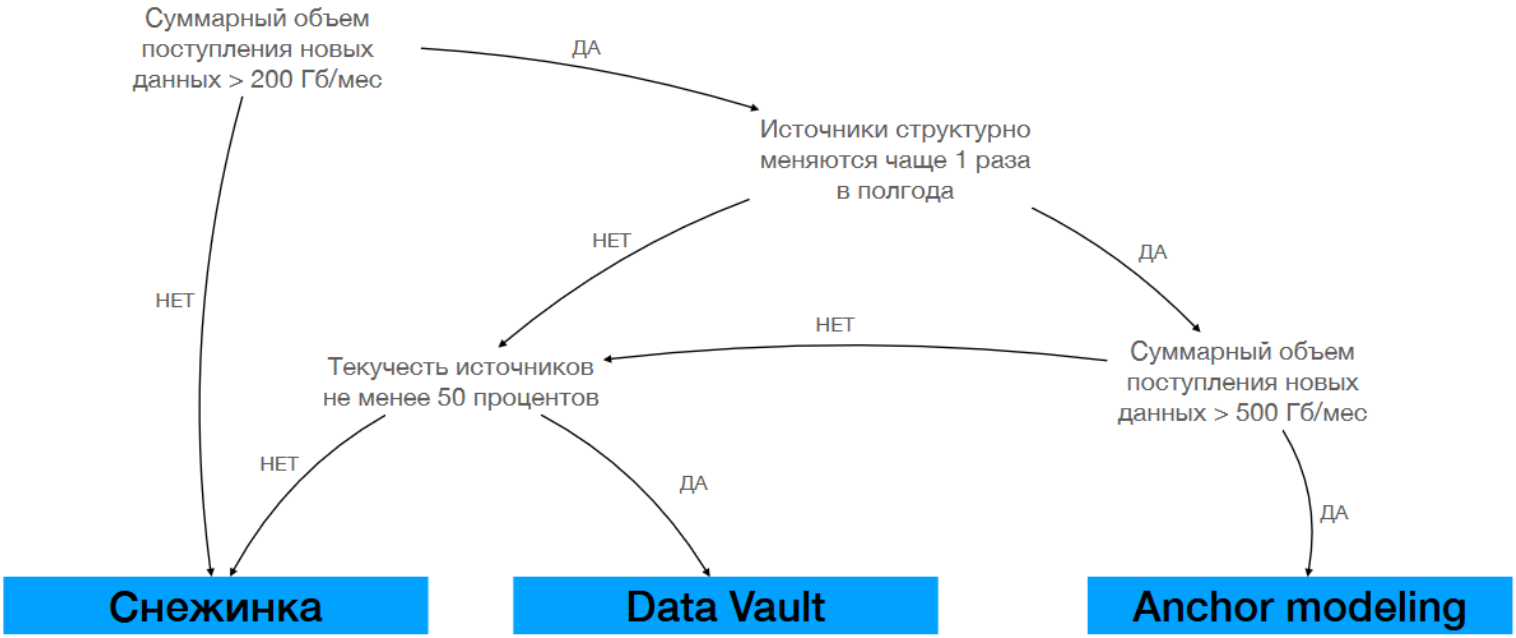
**Плюсы Anchor Modeling:**

- Нормализованная модель, которая эффективно обрабатывает изменения и позволяет масштабировать хранилища данных без отмены предыдущих действий
- Можно независимо разрабатывать смежные источники, так как данные почти полностью отвязаны друг от друга
- Значительная экономия места в связи с отсутствием нулевых значений (null) и дублирования

**Минусы Anchor Modeling:**

- Создает высокую нагрузку на базу даже для основных видов запросов
- Сложно настроить проверку данных, так как модель состоит из большого количества сущностей со сложными связями
- Большое количество таблиц и join'ов, повышающих риск ошибок
- Требует постоянной поддержки и документирования, так как документация уникальна для каждого бизнеса. Специалистам необходимы дополнительные знания для понимания документации.



-- Предикат сравнения (=, <>, >, <, >=, <=)
-- Используется для сравнения значений
-- Пример: найти техников с опытом работы больше 2 лет

```
SELECT firstname, lastname
FROM technician
WHERE hire_date <= CURRENT_DATE - INTERVAL '2 years';
```

-- **BETWEEN** - проверяет, что значение находится в заданном диапазоне (включая границы)
-- Пример: найти заказы, созданные в первом квартале 2024 года

```
SELECT id, created_at
FROM repair_order
WHERE created_at BETWEEN '2024-01-01' AND '2024-03-31';
```

-- **LIKE** - поиск по шаблону
-- % - любое количество любых символов
-- _ - один любой символ
-- Пример: найти сервисные центры, в названии которых есть слово "Tech"

```
SELECT name
FROM service_center
WHERE name LIKE '%Tech%';
```

-- **IN с подзапросом** - проверяет, входит ли значение в результат подзапроса
-- Пример: найти все заказы клиентов из Москвы

```
SELECT id, created_at
FROM repair_order
WHERE client_id IN (
   SELECT id
   FROM client
   WHERE address LIKE 'Moscow%'
);
```

-- **EXISTS** - проверяет, возвращает ли подзапрос какие-либо строки
-- Пример: найти клиентов, у которых есть хотя бы один заказ

```
SELECT firstname, lastname
FROM client c
WHERE EXISTS (
   SELECT 1
   FROM repair_order
   WHERE client_id = c.id
);
```

-- Предикат с квантором (**ALL, ANY, SOME**) - сравнение значения со всеми/любым результатом подзапроса
-- Пример: найти устройства дороже любого ремонта принтера

```
SELECT id, cost
FROM repair_order
WHERE cost > ALL (
   SELECT cost
   FROM repair_order
   WHERE device_type = 'printer'
);
```

-- Агрегатные функции (**COUNT, SUM, AVG, MAX, MIN**) - вычисляют одно значение для группы строк
-- Пример: средняя стоимость ремонта по типам устройств

```
SELECT device_type,
      COUNT(*) as repairs_count,
      AVG(cost) as avg_cost
FROM repair_order
GROUP BY device_type;
```

-- **Скалярные подзапросы** - подзапросы, возвращающие одно значение
-- Пример: добавить к заказу информацию о среднем времени ремонта

```
SELECT id,
       (SELECT AVG(actual_completion - created_at)
        FROM repair_order
        WHERE device_type = r.device_type) as avg_repair_time
FROM repair_order r;
```

-- **CASE** - условное выражение
-- Поисковый CASE (множество условий)
-- Пример: классифицировать стоимость ремонта

```
SELECT id,
       CASE
          WHEN cost < 100 THEN 'Low'
          WHEN cost < 500 THEN 'Medium'
          ELSE 'High'
       END as cost_category
FROM repair_order;
```

-- Простой CASE (сравнение одного выражения)
-- Пример: определить статус заказа более подробно

```
SELECT id,
       CASE status
          WHEN 'pending' THEN 'Waiting for technician'
          WHEN 'in_progress' THEN 'Under repair'
          WHEN 'completed' THEN 'Ready for pickup'
          ELSE status::text
       END as detailed_status
FROM repair_order;
```

-- **JOIN** - объединение таблиц по условию
-- Синтаксис: SELECT ... FROM table1 JOIN table2 ON условие_связи
-- **INNER JOIN** (или просто JOIN) - только строки, имеющие совпадения в обеих таблицах
-- **LEFT JOIN** - все строки из левой таблицы + совпадающие из правой (NULL для несовпадающих)
-- **RIGHT JOIN** - все строки из правой таблицы + совпадающие из левой (NULL для несовпадающих)
-- **SELF JOIN** - соединение таблицы самой с собой (требует алиасы)

-- Пример INNER JOIN: получить все заказы с именами клиентов

```
SELECT r.id, c.firstname, c.lastname
FROM repair_order r
JOIN client c ON r.client_id = c.id;
```

-- Пример LEFT JOIN: получить всех техников и их заказы (включая техников без заказов)

```
SELECT t.firstname, t.lastname, r.id as order_id
FROM technician t
LEFT JOIN repair_order r ON r.technician_id = t.id;
```

-- Пример RIGHT JOIN: получить все заказы и сервисные центры (включая заказы без центра)

```
SELECT s.name, r.id as order_id
FROM service_center s
RIGHT JOIN repair_order r ON r.service_center_id = s.id;
```

-- Пример **SELF JOIN**: найти сервисные центры в одном городе

```
SELECT s1.name, s2.name, s1.address
FROM service_center s1
JOIN service_center s2 ON
   SPLIT_PART(s1.address, ',', 1) = SPLIT_PART(s2.address, ',', 1)
   AND s1.id < s2.id;
```

-- **GROUP BY** - группировка строк с одинаковыми значениями
-- Синтаксис: SELECT столбцы_группировки, агрегатные_функции FROM ... GROUP BY столбцы_группировки
-- Требования:
-- 1. Все неагрегированные столбцы в SELECT должны быть в GROUP BY
-- 2. WHERE фильтрует до группировки, HAVING - после
-- Пример: количество заказов и средняя стоимость по брендам устройств

```
SELECT brand,
    COUNT(*) as orders_count,
    AVG(cost) as avg_cost
FROM repair_order
GROUP BY brand;
```

-- **HAVING** - фильтрация после группировки
-- Синтаксис: ... **GROUP BY ... HAVING** условие_для_сгруппированных_данных
-- Пример: бренды с более чем 10 заказами и средней стоимостью выше 200

```
SELECT brand,
    COUNT(*) as orders_count,
    AVG(cost) as avg_cost
FROM repair_order
GROUP BY brand
HAVING COUNT(*) > 10 AND AVG(cost) > 200;
```

-- **PARTITION BY** - разделение результатов на группы без объединения строк
-- Используется в оконных функциях
-- Синтаксис: оконная_функция OVER (PARTITION BY столбец)
-- Пример: для каждого заказа показать его стоимость и среднюю стоимость по бренду

```
SELECT id,
    brand,
    cost,
    AVG(cost) OVER (PARTITION BY brand) as brand_avg_cost
FROM repair_order;
```

-- Можно комбинировать с ORDER BY внутри окна
-- Пример: пронумеровать заказы каждого клиента по дате

```
SELECT id,
    client_id,
    created_at,
    ROW_NUMBER() OVER (
        PARTITION BY client_id
        ORDER BY created_at
    ) as client_order_number
FROM repair_order;
```

**-- 1. ФУНКЦИИ**

-- Скалярная функция - возвращает одно значение
-- Пример: подсчет общей суммы заказов клиента

```
CREATE OR REPLACE FUNCTION get_client_total_spent(client_id_param INT)
RETURNS NUMERIC AS $$
BEGIN
   RETURN (
      SELECT COALESCE(SUM(cost), 0)
      FROM repair_order
      WHERE client_id = client_id_param
   );
END;
$$ LANGUAGE plpgsql;
```

-- Табличная функция (подставляемая) - возвращает таблицу через RETURNS TABLE

```
-- Пример: получение всех заказов клиента с информацией о статусе
CREATE OR REPLACE FUNCTION get_client_orders(client_id_param INT)
RETURNS TABLE (
    order_id INT,
    device TEXT,
    status order_status,
    cost NUMERIC
) AS $$
BEGIN
    RETURN QUERY
    SELECT id, device_type::TEXT, status, cost
    FROM repair_order
    WHERE client_id = client_id_param;
END;
$$ LANGUAGE plpgsql;

-- Многооператорная табличная функция - возвращает таблицу через RETURNS SETOF
-- Пример: поиск техников по специализации с подсчетом их заказов
CREATE OR REPLACE FUNCTION find_technicians_by_specialization(device_type_param device_type)
RETURNS SETOF technician AS $$
DECLARE
    tech_record technician;
BEGIN
    FOR tech_record IN
        SELECT t.*
        FROM technician t
        WHERE device_type_param = ANY(t.specialization)
    LOOP
        RETURN NEXT tech_record;
    END LOOP;
    RETURN;
END;
$$ LANGUAGE plpgsql;
```

**-- 2. ХРАНИМЫЕ ПРОЦЕДУРЫ**

```
-- Процедура без параметров
-- Пример: очистка старых отмененных заказов
CREATE OR REPLACE PROCEDURE cleanup_cancelled_orders()
LANGUAGE plpgsql AS $$
BEGIN
    DELETE FROM repair_order
    WHERE status = 'cancelled'
    AND created_at < CURRENT_DATE - INTERVAL '1 year';
    COMMIT;
END;
$$;

-- Процедура с курсором
-- Пример: обновление баллов лояльности для клиентов
CREATE OR REPLACE PROCEDURE update_loyalty_points()
LANGUAGE plpgsql AS $$
DECLARE
    client_cursor CURSOR FOR
        SELECT id, loyalty_points
        FROM client;
    client_record RECORD;
BEGIN
    OPEN client_cursor;
```

```
    LOOP
        FETCH client_cursor INTO client_record;
        EXIT WHEN NOT FOUND;

        UPDATE client
        SET loyalty_points = client_record.loyalty_points + 100
        WHERE id = client_record.id;
    END LOOP;
    CLOSE client_cursor;
END;
$$;
```

**-- 3. ТРИГГЕРЫ**

-- Триггер **AFTER**

```
-- Пример: обновление рейтинга сервисного центра после завершения заказа
CREATE OR REPLACE FUNCTION update_service_center_rating()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'completed' THEN
        UPDATE service_center
        SET rating = (
            SELECT AVG(cost)::REAL
            FROM repair_order
            WHERE service_center_id = NEW.service_center_id
            AND status = 'completed'
        )
        WHERE id = NEW.service_center_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER repair_completed_trigger
AFTER UPDATE ON repair_order
FOR EACH ROW
WHEN (NEW.status = 'completed' AND OLD.status != 'completed')
EXECUTE FUNCTION update_service_center_rating();
```

-- Триггер **INSTEAD OF**

```
-- Пример: логирование попыток удаления заказов
CREATE VIEW repair_order_view AS
SELECT * FROM repair_order;

CREATE OR REPLACE FUNCTION log_delete_attempt()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO delete_log (order_id, delete_time)
    VALUES (OLD.id, CURRENT_TIMESTAMP);
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER prevent_delete_trigger
INSTEAD OF DELETE ON repair_order_view
FOR EACH ROW
EXECUTE FUNCTION log_delete_attempt();
```

Ключевые особенности:

1. Функции обязательно должны что-то возвращать (RETURNS)
2. Процедуры не возвращают значений
3. Триггеры привязываются к событиям таблицы (INSERT/UPDATE/DELETE)
4. AFTER триггеры выполняются после события
5. INSTEAD OF триггеры заменяют собой событие
6. Для работы с INSTEAD OF требуются представления (VIEW)

1. Базы данных и системы управления базами данных. Определения, основные функции и классификация
2. Семантическое моделирование данных
3. Реляционная модель данных: структурная, целостная, манипуляционная части. Реляционная алгебра. Исчисление кортежей
4. Теория проектирования реляционных баз данных: функциональные зависимости, нормальные формы
5. Теория проектирования хранилищ данных. Основные принципы построения. ETL и ELT процессы
6. Транзакции. Определение, свойства и уровни изоляции транзакций. Неблагоприятные эффекты, вызванные параллельным выполнением транзакций, и способы их устранения. Управление транзакциями и способы обработки ошибок
7. Блокировки. Определение, свойства, иерархии, гранулярность и взаимоблокировки, алгоритмы обнаружения взаимоблокировок
8. Журнализация. Операции журнала транзакций и его логическая и физическая архитектуры. Модели восстановления. Метаданные
9. Безопасность и Аудит. Ключевые понятия и участники системы безопасности. Модели управления доступом
10. MPP системы. Распределенное и колоночное хранение. Распределенные вычисления, модель MapReduce. Обеспечение отказоустойчивости.
11. In-Memory базы данных. Преимущества и недостатки. Примеры использования
12. Инструкции языка описания данных, инструкции языка обработки данных, инструкции безопасности, инструкции управления транзакциями
13. Объекты базы данных: функции, процедуры, триггеры и курсоры
14. Оптимизация запроса: индексы, партиционирование, сегментирование
15. План запроса. Этапы выполнения запроса
16. Гибкие методологии проектирования. Data Vault, Anchor Modeling